

```
# -*- coding: utf-8 -*-
import os
import re
import socket
import subprocess
from libqtile import qtile
from libqtile.config import Click, Drag, Group, KeyChord, Key, Match, Screen
from libqtile.lazy import lazy
from libqtile import layout, bar, widget, hook
from libqtile.lazy import lazy
from libqtile.utils import guess_terminal
from typing import List # noqa: F401

from qtile_extras import widget
from qtile_extras.widget.decorations import BorderDecoration

mod = "mod4" # Sets mod key to SUPER/WINDOWS
myTerm = "alacritty" # My terminal of choice
myBrowser = "google-chrome-stable" # My browser of choice

keys = [
    ### The essentials
    Key([mod], "Return",
        lazy.spawn(myTerm),
        desc='Launches My Terminal'
    ),
    Key([mod, "shift"], "Return",
        lazy.spawn("rofi -show drun"),
        desc='Run Launcher'
    ),
    Key([mod, "shift"], "p",
        lazy.spawn("rofi -show run"),
        desc='Run Show Launcher'
    ),
    Key([mod], "e",
        lazy.spawn("emacsclient -nc"),
        desc='open emacs'
    ),
    Key([mod, "shift"], "e",
        lazy.spawn("rofi -show emoji"),
        desc='emoji'
    ),
    Key([mod], "b",
        lazy.spawn("rofi-bluetooth"),
        desc='bluetooth'
    ),
    Key([mod, "shift"], "w",
        lazy.spawn("/home/fred/.local/bin/wifimenu"),
        desc='Wifi Selector'
    ),
    Key([mod], "x",
        lazy.spawn("clipmenu"),
        desc='clipmenu'
    ),
    Key([mod, "shift"], "s",
        lazy.spawn("flameshot gui"),
        desc='flameshot'
    ),
    Key([mod, "shift"], "b",
        lazy.spawn(myBrowser),
        desc='Google Chrome'
    ),
    Key([mod], "Tab",
```

```
    lazy.next_layout(),
    desc='Toggle through layouts'
),
Key([mod, "shift"], "c",
    lazy.window.kill(),
    desc='Kill active window'
),
Key([mod, "shift"], "r",
    lazy.restart(),
    desc='Restart Qtile'
),
Key([mod, "shift"], "q",
    lazy.spawn("rofi -show power-menu -modi power-menu:/home/fred/.local/bin/powermenu"),
    desc='powermenu'
),
Key([mod, "shift"], "f",
    lazy.spawn("alacritty -e ranger"),
    desc='Ranger'
),
Key([mod, "shift"], "m",
    lazy.spawn("messenger-nativefier"),
    desc='messenger'
),
Key([mod, "shift"], "x",
    lazy.spawn("rofi -show calc -modi calc - no-show-match -no-sort"),
    desc='calc'
),
### Window controls
Key([mod], "j",
    lazy.layout.down(),
    desc='Move focus down in current stack pane'
),
Key([mod], "k",
    lazy.layout.up(),
    desc='Move focus up in current stack pane'
),
Key([mod, "shift"], "j",
    lazy.layout.shuffle_down(),
    lazy.layout.section_down(),
    desc='Move windows down in current stack'
),
Key([mod, "shift"], "k",
    lazy.layout.shuffle_up(),
    lazy.layout.section_up(),
    desc='Move windows up in current stack'
),
Key([mod], "h",
    lazy.layout.shrink(),
    lazy.layout.decrease_nmaster(),
    desc='Shrink window (MonadTall), decrease number in master pane (Tile)'
),
Key([mod], "l",
    lazy.layout.grow(),
    lazy.layout.increase_nmaster(),
    desc='Expand window (MonadTall), increase number in master pane (Tile)'
),
Key([mod], "n",
    lazy.layout.normalize(),
    desc='normalize window size ratios'
),
Key([mod], "m",
    lazy.layout.maximize(),
    desc='toggle window between minimum and maximum sizes'
```

```

    ),
    Key(["control", "shift"], "f",
        lazy.window.toggle_floating(),
        desc='toggle floating'
    ),
    Key([mod], "f",
        lazy.window.toggle_fullscreen(),
        desc='toggle fullscreen'
    ),
    ### Stack controls
    Key([mod, "shift"], "Tab",
        lazy.layout.rotate(),
        lazy.layout.flip(),
        desc='Switch which side main pane occupies (XmonadTall)'
    ),
    Key([mod], "space",
        lazy.layout.next(),
        desc='Switch window focus to other pane(s) of stack'
    ),
    Key([mod, "shift"], "space",
        lazy.layout.toggle_split(),
        desc='Toggle between split and unsplit sides of stack'
    ),
]

```

```

groups = [Group("Alacritty", layout='monadtall', matches=[Match(wm_class=["Alacritty"])]),
           Group("Chrome", layout='monadtall', matches=[Match(wm_class=["Google-chrome"])]),
           Group("Chat", layout='monadtall', matches=[Match(wm_class=["googlemessages-
nativefier-11f104", "facebookmessenger-nativefier-7ab88e"])]),
           Group("Music", layout='monadtall', matches=[Match(wm_class=["Spotify"])]),
           Group("Video", layout='monadtall', matches=[Match(wm_class=["Plex"])]),
           Group("Email", layout='monadtall', matches=[Match(wm_class=["org.mozilla.Thunderbird"])]),
           Group("Code", layout='monadtall', matches=[Match(wm_class=["Code", "Emacs"])]),
           Group("8", layout='monadtall', matches=[Match(wm_class=["vibe-typer"])]),
           Group("9", layout='monadtall'),
           Group("10", layout='monadtall')]

```

```

# Allow MODKEY+[0 through 9] to bind to groups, see https://docs.qtile.org/en/stable/manual/config/groups.html
# MOD4 + index Number : Switch to Group[index]
# MOD4 + shift + index Number : Send active window to another Group
from libqtile.dgroups import simple_key_binder
dgroups_key_binder = simple_key_binder("mod4")

```

```

layout_theme = {"border_width": 2,
                "margin": 8,
                "border_focus": "e1acff",
                "border_normal": "1D2330"
                }

```

```

layouts = [
    #layout.MonadWide(**layout_theme),
    #layout.Bsp(**layout_theme),
    #layout.Stack(stacks=2, **layout_theme),
    #layout.Columns(**layout_theme),
    #layout.RatioTile(**layout_theme),
    #layout.Tile(shift_windows=True, **layout_theme),
    #layout.VerticalTile(**layout_theme),
    #layout.Matrix(**layout_theme),
    #layout.Zoomy(**layout_theme),
    layout.MonadTall(**layout_theme),
    layout.Max(**layout_theme),
    layout.Stack(num_stacks=2),
    layout.RatioTile(**layout_theme),
]

```

```

layout.TreeTab(
    font = "Ubuntu",
    fontsize = 10,
    sections = ["FIRST", "SECOND", "THIRD", "FOURTH"],
    section_fontsize = 10,
    border_width = 2,
    bg_color = "1c1f24",
    active_bg = "c678dd",
    active_fg = "000000",
    inactive_bg = "a9a1e1",
    inactive_fg = "1c1f24",
    padding_left = 0,
    padding_x = 0,
    padding_y = 5,
    section_top = 10,
    section_bottom = 20,
    level_shift = 8,
    vspace = 3,
    panel_width = 200
),
layout.Floating(**layout_theme)
]

def init_colors():
    return [
        ["#282c34", "#282c34"],
        ["#1c1f24", "#1c1f24"],
        ["#dfdfdf", "#dfdfdf"],
        ["#ff6c6b", "#ff6c6b"],
        ["#98be65", "#98be65"],
        ["#da8548", "#da8548"],
        ["#51afef", "#51afef"],
        ["#c678dd", "#c678dd"],
        ["#46d9ff", "#46d9ff"],
        ["#a9a1e1", "#a9a1e1"]
    ]

colors = init_colors()

prompt = "{0}@{1}: ".format(os.environ["USER"], socket.gethostname())

##### DEFAULT WIDGET SETTINGS #####
widget_defaults = dict(
    font="Ubuntu Bold",
    fontsize = 10,
    padding = 2,
    background=colors[2]
)
extension_defaults = widget_defaults.copy()

def init_widgets_list():
    widgets_list = [
        widget.Sep(
            linewidth = 0,
            padding = 6,
            foreground = colors[2],
            background = colors[0]
        ),
        widget.Image(
            filename = "~/.config/qtile/icons/python-white.png",
            scale = "False",
            mouse_callbacks = {'Button1': lambda: qtile.cmd_spawn(myTerm)}
        ),
        widget.Sep(
            linewidth = 0,

```

```
        padding = 6,
        foreground = colors[2],
        background = colors[0]
    ),
    widget.GroupBox(
        font = "Ubuntu Bold",
        fontsize = 9,
        margin_y = 3,
        margin_x = 0,
        padding_y = 5,
        padding_x = 3,
        borderwidth = 3,
        active = colors[2],
        inactive = colors[7],
        rounded = False,
        highlight_color = colors[1],
        highlight_method = "line",
        this_current_screen_border = colors[6],
        this_screen_border = colors[4],
        other_current_screen_border = colors[6],
        other_screen_border = colors[4],
        foreground = colors[2],
        background = colors[0]
    ),
    widget.TextBox(
        text = '|',
        font = "Ubuntu Mono",
        background = colors[0],
        foreground = '474747',
        padding = 2,
        fontsize = 14
    ),
    widget.CurrentLayoutIcon(
        custom_icon_paths = [os.path.expanduser("~/config/qtile/icons")],
        foreground = colors[2],
        background = colors[0],
        padding = 0,
        scale = 0.7
    ),
    widget.CurrentLayout(
        foreground = colors[2],
        background = colors[0],
        padding = 5
    ),
    widget.TextBox(
        text = '|',
        font = "Ubuntu Mono",
        background = colors[0],
        foreground = '474747',
        padding = 2,
        fontsize = 14
    ),
    widget.WindowName(
        foreground = colors[6],
        background = colors[0],
        padding = 0
    ),
    widget.Sep(
        linewidth = 0,
        padding = 6,
        foreground = colors[0],
        background = colors[0]
    ),
```

```
widget.Net(  
    interface = "wlan0",  
    format = 'Net: {down} ↓↑ {up}',  
    foreground = colors[3],  
    background = colors[0],  
    padding = 5,  
    decorations=[  
        BorderDecoration(  
            colour = colors[3],  
            border_width = [0, 0, 2, 0],  
            padding_x = 5,  
            padding_y = None,  
        )  
    ],  
),  
widget.Sep(  
    linewidth = 0,  
    padding = 6,  
    foreground = colors[0],  
    background = colors[0]  
),  
widget.ThermalSensor(  
    foreground = colors[3],  
    background = colors[0],  
    threshold = 90,  
    fmt = 'Temp: {}',  
    padding = 5,  
    decorations=[  
        BorderDecoration(  
            colour = colors[3],  
            border_width = [0, 0, 2, 0],  
            padding_x = 5,  
            padding_y = None,  
        )  
    ],  
),  
widget.Sep(  
    linewidth = 0,  
    padding = 6,  
    foreground = colors[0],  
    background = colors[0]  
),  
widget.DF(  
    visible_on_warn = False,  
    background = colors[0],  
    foreground = colors[4],  
    font = "Ubuntu Bold",  
    decorations = [  
        BorderDecoration (  
            colour = colors[4],  
            border_width = [0,0,2,0],  
            padding_x = 5,  
            radius = 2,  
            filled = True  
        )  
    ],  
),  
widget.Sep(  
    linewidth = 0,  
    padding = 6,  
    foreground = colors[0],  
    background = colors[0]  
),
```

```
widget.DF(  
    visible_on_warn = False,  
    background = colors[0],  
    foreground = colors[5],  
    font = "Ubuntu Bold",  
    partition = "/home",  
    decorations = [  
        BorderDecoration(  
            colour = colors[5],  
            border_width = [0,0,2,0],  
            padding_x = 5,  
            radius = 2,  
            filled = True  
        ),  
    ],  
),  
widget.Sep(  
    linewidth = 0,  
    padding = 6,  
    foreground = colors[0],  
    background = colors[0]  
),  
widget.Memory(  
    foreground = colors[9],  
    background = colors[0],  
    mouse_callbacks = {'Button1': lambda: qtile.cmd_spawn(myTerm + ' -e htop')},  
    fmt = 'Mem: {}',  
    padding = 5,  
    decorations=[  
        BorderDecoration(  
            colour = colors[9],  
            border_width = [0, 0, 2, 0],  
            padding_x = 5,  
            padding_y = None,  
        )  
    ],  
),  
widget.Sep(  
    linewidth = 0,  
    padding = 6,  
    foreground = colors[0],  
    background = colors[0]  
),  
widget.Volume(  
    foreground = colors[7],  
    background = colors[0],  
    fmt = 'Vol: {}',  
    padding = 5,  
    decorations=[  
        BorderDecoration(  
            colour = colors[7],  
            border_width = [0, 0, 2, 0],  
            padding_x = 5,  
            padding_y = None,  
        )  
    ],  
),  
widget.Sep(  
    linewidth = 0,  
    padding = 6,  
    foreground = colors[0],
```

```

        background = colors[0]
    ),
    widget.Clock(
        foreground = colors[6],
        background = colors[0],
        format = "  %A, %B %d - %I:%M %p ",
        decorations=[
            BorderDecoration(
                colour = colors[6],
                border_width = [0, 0, 2, 0],
                padding_x = 5,
                padding_y = None,
            )
        ],
    ),
    widget.Sep(
        linewidth = 0,
        padding = 6,
        foreground = colors[0],
        background = colors[0]
    ),
    widget.UPowerWidget(
        background = colors[0],
        border_colour = '#d8dee9',
        border_critical_colour = '#bf616a',
        padding_x = 5
    ),
    widget.Sep(
        linewidth = 0,
        padding = 6,
        foreground = colors[0],
        background = colors[0]
    ),
    widget.Systray(
        background = colors[0],
        padding = 5
    )
]
return widgets_list

widgets_list = init_widgets_list()

def init_widgets_screen1():
    return init_widgets_list()

widgets_screen1 = init_widgets_screen1()

def init_screens():
    return [Screen(top=bar.Bar(widgets=init_widgets_screen1(), size=28, opacity=0.97))]
screens = init_screens()

def window_to_prev_group(qtile):
    if qtile.currentWindow is not None:
        i = qtile.groups.index(qtile.currentGroup)
        qtile.currentWindow.togroup(qtile.groups[i - 1].name)

def window_to_next_group(qtile):
    if qtile.currentWindow is not None:
        i = qtile.groups.index(qtile.currentGroup)

```



```

        qtile.currentWindow.togroup(qtile.groups[i + 1].name)

mouse = [
    Drag([mod], "Button1", lazy.window.set_position_floating(),
        start=lazy.window.get_position()),
    Drag([mod], "Button3", lazy.window.set_size_floating(),
        start=lazy.window.get_size()),
    Click([mod], "Button2", lazy.window.bring_to_front())
]

dgroups_app_rules = [] # type: List
follow_mouse_focus = True
bring_front_click = False
cursor_warp = False

floating_layout = layout.Floating(float_rules=[
    # Run the utility of `xprop` to see the wm class and name of an X client.
    # default_float_rules include: utility, notification, toolbar, splash, dialog,
    # file_progress, confirm, download and error.
    *layout.Floating.default_float_rules,
    Match(title='Confirmation'),      # tastyworks exit box
    Match(title='Qalculate!'),        # qalculate-gtk
])
auto_fullscreen = True
focus_on_window_activation = "smart"
reconfigure_screens = True

# If things like steam games want to auto-minimize themselves when losing
# focus, should we respect this or not?
auto_minimize = True

@hook.subscribe.startup_once
def start_once():
    home = os.path.expanduser('~')
    subprocess.call([home + '/.config/qtile/autostart.sh'])

# XXX: Gasp! We're lying here. In fact, nobody really uses or cares about this
# string besides java UI toolkits; you can see several discussions on the
# mailing lists, GitHub issues, and other WM documentation that suggest setting
# this string if your java app doesn't work correctly. We may as well just lie
# and say that we're a working one by default.
#
# We choose LG3D to maximize irony: it is a 3D non-reparenting WM written in
# java that happens to be on java's whitelist.
wmname = "LG3D"

```